Experiment 8

Aim

To implement banking-style transactions using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT in SQL, simulate partial failures, and demonstrate isolation levels and concurrency anomalies using two database sessions.

Database Used

- Database Management System: PostgreSQL
- Query Language: SQL
- Tool: pgAdmin 4 or PostgreSQL command-line interface
- Concepts Used: Transactions, ACID properties, savepoints, isolation levels, concurrency control.

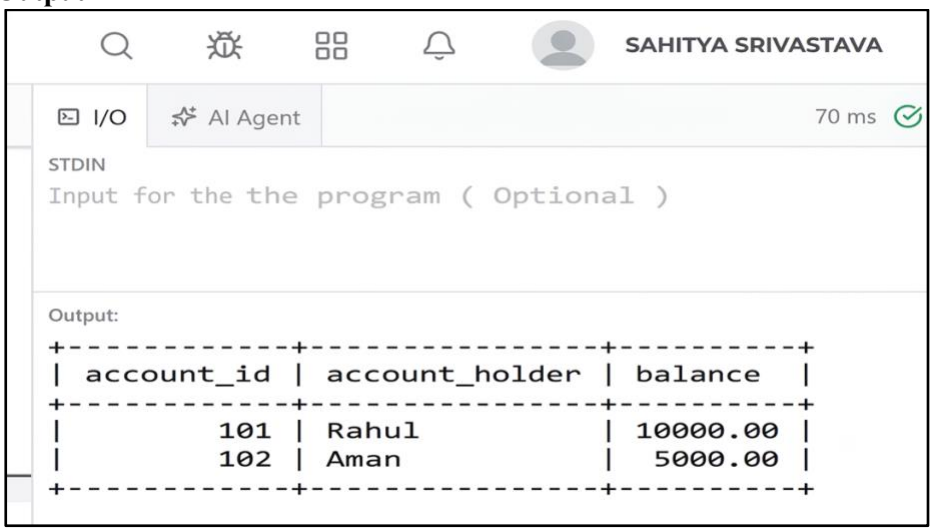
Source Code

A. Create the Bank Accounts Table

```
CREATE TABLE bank_accounts (
   account_id INT PRIMARY KEY,
   account_holder VARCHAR(50),
   balance NUMERIC(12, 2) CHECK (balance >= 0)
);

INSERT INTO bank_accounts
VALUES
   (101, 'Rahul', 10000.00),
   (102, 'Aman', 5000.00);
SELECT * FROM bank_accounts;
```

Output

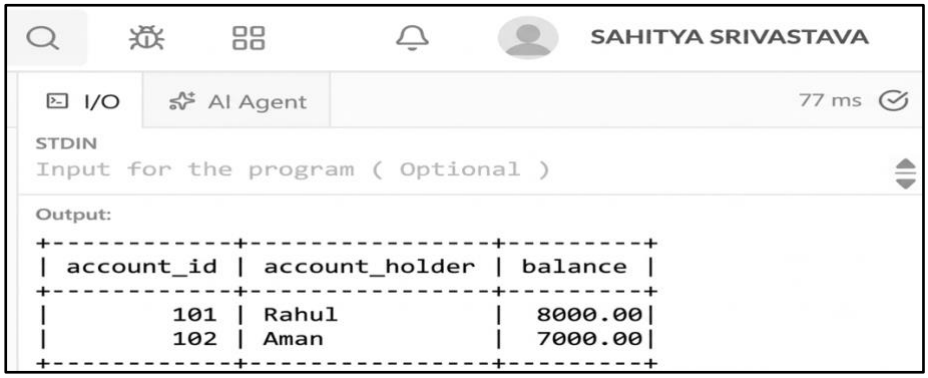


B. Demonstrate BEGIN and COMMIT

Transfer ₹2,000 from Rahul's account to Aman's account. Both operations should succeed together.

```
BEGIN;
UPDATE bank_accounts
SET balance = balance - 2000
WHERE account_id = 101;
UPDATE bank_accounts
SET balance = balance + 2000
WHERE account_id = 102;
COMMIT;
SELECT * FROM bank_accounts;
```

Output



C. Demonstrate ROLLBACK

Withdraw ₹1,000, but cancel the transaction before committing.

```
BEGIN;
UPDATE bank_accounts
SET balance = balance - 1000
WHERE account_id = 101;
SELECT * FROM bank_accounts;
ROLLBACK;
SELECT * FROM bank_accounts;
```

Output



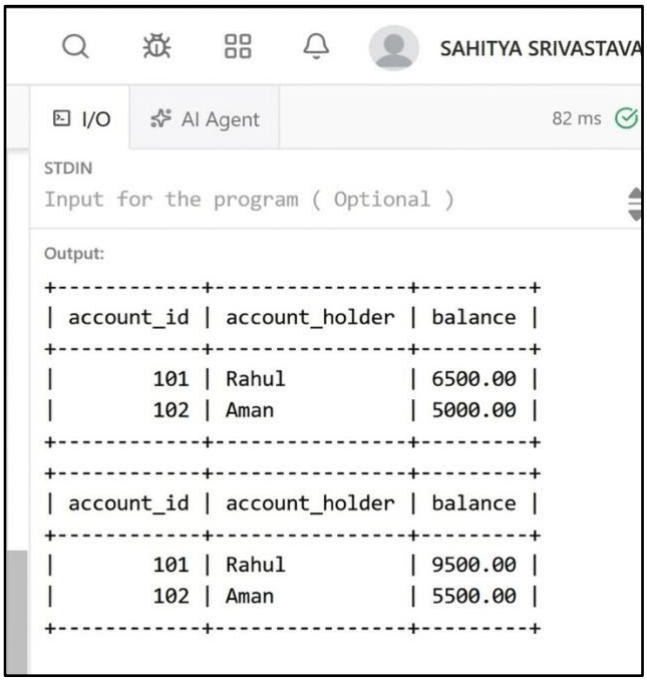
D. Demonstrate SAVEPOINT

Suppose a transaction contains multiple operations, but one operation needs to be undone without cancelling the entire transaction.

```
BEGIN;
UPDATE bank_accounts
SET balance = balance - 500
WHERE account_id = 101;
SAVEPOINT sp1;
UPDATE bank_accounts
SET balance = balance - 20000
WHERE account_id = 101;
ROLLBACK TO SAVEPOINT sp1;

UPDATE bank_accounts
SET balance = balance + 500
WHERE account_id = 102;
COMMIT;
SELECT * FROM bank_accounts;
```

Output



Explanation: SAVEPOINT sp1 marks a point inside the transaction. ROLLBACK TO SAVEPOINT sp1 undoes the subsequent changes while keeping earlier changes. The final COMMIT saves the remaining operations.

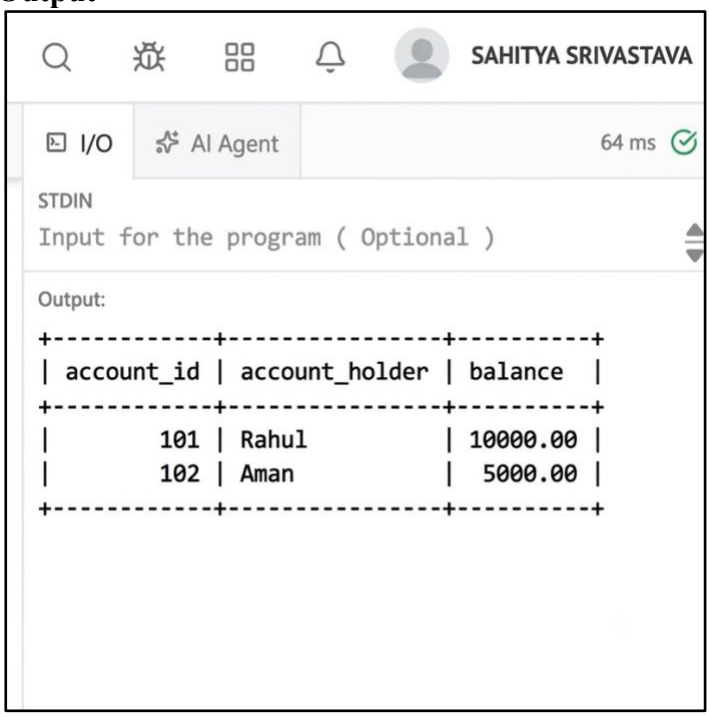
E. Demonstrate Transaction Isolation

The following query sets the isolation level for a transaction.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
START TRANSACTION;

SELECT * FROM bank_accounts;
COMMIT;
```

Output



Explanation: SERIALIZABLE is PostgreSQL's strictest standard transaction isolation level. It aims to make concurrent transactions behave as though they were executed sequentially. PostgreSQL may abort a transaction if concurrent operations cannot safely be serialized.

F. Demonstrate Concurrency Anomalies Using Two Sessions

Open two separate query windows in pgAdmin, called Session 1 and Session 2.

```
-- Session 1:

BEGIN;

UPDATE bank_accounts

SET balance = 9000

WHERE account_id = 101;

-- Do not commit yet.
```




```
-- Session 2:
BEGIN TRANSACTION
ISOLATION LEVEL READ COMMITTED;
SELECT balance
FROM bank_accounts
WHERE account_id = 101;
COMMIT;
```



Explanation: Session 2 will normally see the last committed balance, not the uncommitted value 9000 from Session 1. This demonstrates how PostgreSQL prevents dirty reads.

Result

The experiment successfully demonstrates transaction management in PostgreSQL using BEGIN, COMMIT, ROLLBACK, and SAVEPOINT. It also illustrates isolation levels and concurrency behavior through two database sessions.

Conclusion

This experiment demonstrates how database transactions maintain data consistency during banking operations. COMMIT saves changes, ROLLBACK cancels uncommitted changes, and SAVEPOINT allows partial recovery.
Transaction isolation levels help control the effects of concurrent transactions and prevent anomalies such as dirty reads. These mechanisms are essential for reliable banking and financial applications.